Case Study on the Analysis of SDLC Models Applied to AI Lecture Assistant Application

Student Name: Krishnaprasad.S | Roll Number: 241201026

Aim

To analyze the Software Development Life Cycle (SDLC) models used in the development of an **AI Lecture Assistant Application** and identify the most suitable SDLC model.

Objective

- Study different SDLC models.
- Analyze the development process of an AI-powered lecture recording and summary application.
- Compare the advantages and limitations of various SDLC models.
- Recommend the most appropriate SDLC model for the application.

Selected Software Application

AI Lecture Assistant System

An AI Lecture Assistant Application is designed to record live classroom or virtual lectures, transcribe audio into structured notes, and automatically highlight and generate key questions based on emphasis points detected during the lecture.

Functional Requirements

Student / User Module

- **Lecture Recording:** Live audio/video recording and file upload.
- **Automated Note Generation:** Speech-to-text transcription transformed into structured notes.
- **Important Questions Generator:** Natural Language Processing (NLP) detection of highlighted topics, repeated phrases, or instructor cues to generate key questions.
- **Dashboard & Repository:** Store, categorize, and search through past lecture notes and generated questions.
- **Export & Sharing:** Export generated notes as PDF/Text and share with peers.

AI Processing Engine Module

- **Audio Processing & Speech-to-Text:** Converts speech into written text in real time or post-recording.
- **Keyword & Emphasis Detection:** Highlights key concepts, definitions, and teacher-emphasized topics.
- **Summarization & Question Mining:** Parses text to generate study summaries and potential exam questions.

Admin / System Module

- **User Management:** Profile creation and access management.
- **Model Training & Updates:** Periodic updating of speech recognition and NLP extraction models.
- **Analytics & Usage Logs:** Monitor storage usage, processing load, and app performance.

SDLC Models Analysis

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Simple, clear phase boundaries, and easy to document.	Inflexible; poorly handles evolving AI/NLP requirements.
Prototype	High	Helps refine user UI/UX and audio processing accuracy early.	May lead to excessive redesign iterations.
Spiral	High	Excellent risk management for complex machine learning tasks.	Expensive, complex, and slow for early MVPs.
Agile	Very High	Iterative updates based on speech recognition accuracy and user feedback.	Requires active user collaboration and continuous refinement.
DevOps	Very High	Continuous Integration/Continuous Deployment (CI/CD) for app and AI models.	High initial pipeline setup and infrastructure complexity.

Why Agile is Best?

- **AI Model Accuracy Refinement:** Speech recognition and question extraction require continuous tuning based on real-world classroom audio quality and instructor accents.
- **Evolving User Needs:** Student features (e.g., flashcard generation, key phrase highlighting, quiz mode) can be introduced incrementally.
- **Rapid Bug Fixes & Updates:** Mobile audio capture updates and transcription bug fixes must be rolled out quickly.
- **Iterative Feedback Loops:** Direct feedback from students and educators helps tune the NLP model's ability to identify what teachers consider "important."

Agile Development Process

> **Requirement Gathering ↓ Sprint Planning ↓ Design (UI & AI Architecture) ↓ Development (Audio & NLP) ↓ Testing (Speech & Questions) ↓ Deployment ↓ Student Feedback ↓ Next Sprint**

SDLC Phases

1. Requirement Analysis

- **Functional Requirements:** Audio recording quality, NLP summary generation, teacher emphasis detection algorithm.
- **Non-Functional Requirements:** Low processing latency, high transcription accuracy, secure cloud storage.
- **Deliverables:** Software Requirement Specification (SRS) Document.

2. System Design

- **Design Elements:** Audio Pipeline Architecture, Database Schema, UI/UX Wireframes, AI/NLP API Integration.
- **Deliverables:** UML Class and Sequence Diagrams, System Architecture Diagram.

3. Development

- **Modules Developed:** Audio Capture Module, Speech-to-Text & Summarization Engine, Question Generation Engine, Notes Dashboard.
- **Tech Stack:** Python (AI/NLP processing), React Native / Flutter (Mobile UI), Node.js (Backend API).
- **Database & Cloud:** PostgreSQL / MongoDB, AWS S3.

4. Testing

- **Testing Types:** Unit Testing, Integration Testing, Model Performance Testing, User Acceptance Testing (UAT).

5. Deployment

- **Deployment Options:** Cloud Server Deployment (Dockerized), Mobile App Store (iOS/Android Release).
- **Cloud Platforms:** AWS / Google Cloud / Azure.

6. Maintenance

- **Activities:** AI Model re-training for improved domain terminology, security updates, feature enhancements, and system scaling.

Comparison of SDLC Models

Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Requirement Changes	Poor	Good	Excellent	Excellent	Excellent
Customer Feedback	Low	High	High	Very High	Very High
Risk Handling	Low	Medium	High	Medium	High
Delivery Speed	Slow	Medium	Medium	Fast	Very Fast
Cost	Low	Medium	High	Medium	Medium
Suitable for AI App	No	Partially	Yes	Best	Best

Results

- **Iterative Refinement:** The AI Lecture Assistant Application requires continuous feature updates, model retraining, and real-time user feedback.
- **Agile Benefits:** Agile SDLC supports iterative development, rapid releases, and flexibility.
- **DevOps Synergy:** DevOps complements Agile by enabling continuous integration, automated model testing, and seamless deployment.
- **Recommendation:** Therefore, Agile with DevOps practices is the most suitable approach for developing and maintaining the application.

Conclusion

The case study demonstrates that selecting the appropriate SDLC model depends heavily on application dynamics and user expectations. For an AI-driven application such as the AI Lecture Assistant, Agile provides the necessary flexibility for iterative enhancements in NLP accuracy and UI usability, while DevOps ensures efficient, continuous delivery and system stability. This combined approach guarantees high note generation quality, rapid deployment of updates, and high user satisfaction.